Laboratorio 2 — One size fits all?

TI-4601 Bases de Datos Avanzadas · Grupo 3 · Fabricio Mena · Christopher Rodriguez · Alisson Redondo

1. Entorno y predicciones

Todas las medidas salen de una sola corrida, el 17/10/2026: `cargar.py`, `analitica.py`, `escrituras.py` y `graficas.py` seguidos, desde un `resultados.jsonl` limpio.

Máquina	Laptop AMD Ryzen 7 5700U, 4 CPU asignados a WSL 2
Sistema operativo	Windows 11 Home con Ubuntu 24.04 en WSL 2, Docker Desktop 29.5.2
RAM	16 GB (9,7 GiB para WSL 2)
Motores	Postgres 16 y Valkey 8.1 (`--appendonly no`, `--maxmemory 4gb`)
Perfil	completo: 10 906 858 viajes en Postgres y 1 000 000 en Valkey

Las predicciones están en `evidence/predicciones.md`. Las de los pasos 1 y 3 se escribieron antes de correr cada paso. La del paso 2 se documentó después de correrlo, así que no cuenta como predicción previa; se deja como análisis de lo esperado.

Métrica	**Predicción**	**Medido**
Paso 1: carga de Postgres	∼150 000 filas/s	165 323 filas/s
Paso 1: carga de Valkey	∼30 000 filas/s	19 141 filas/s
Paso 1: bytes por fila, Postgres / Valkey	∼180 / ∼550	181,5 / 549,4
Paso 2: `GROUP BY` (no previa)	gana Postgres	Postgres, 81×
Paso 3a: lectura / update por llave	Valkey ∼1,5× / ∼20×	Valkey 1,08× / 8,8×
Paso 3b: `valkey_aof_always` vs `postgres_commit_por_mensaje`	Valkey ∼1,3×	Postgres 2,0×
Paso 3b: `valkey_pipeline`	∼50 000 msg/s	51 040 msg/s

El espacio salió casi exacto. Fallamos en dos cosas: la carga de Valkey fue más lenta de lo previsto (armar cada `HSET` en Python cuesta más de lo estimado) y con `appendfsync always` Valkey quedó por debajo de Postgres, no por encima.

2. Código de los TODO y verificaciones

TODO 1 (`cargar.py`)

```
    cur.execute(
        "SELECT pg_relation_size('trips'), pg_indexes_size('trips'),"
        " pg_total_relation_size('trips')"
    )
    return cur.fetchone()
```

TODO 2 (`cargar.py`)

```
    pipe = r.pipeline(transaction=False)
    n = 0
    for n, row in enumerate(iter_rows(limit), start=1):
        pipe.hset(f"trip:{n}", mapping=dict(zip(CSV_COLUMNS, row)))
        if n % PIPELINE_BATCH == 0:
            pipe.execute()
    pipe.execute()
    return n
```

TODO 3 (`analitica.py`)

```
    pipe = r.pipeline(transaction=False)
    for k in keys:
        pipe.hmget(k, "payment_type", "fare_amount")
    results = pipe.execute()
    for payment_type, fare_amount in results:
        if payment_type is None or fare_amount is None:
            continue
        pt = str(payment_type)
        fare = float(fare_amount)
        sums[pt] = sums.get(pt, 0.0) + fare
        counts[pt] = counts.get(pt, 0) + 1
```

TODO 4 (`lua/feed.lua`)

```
redis.call('HINCRBY', 'agg:n', pt, 1)
redis.call('HINCRBYFLOAT', 'agg:suma', pt, fare)
```

```
if anom == '1' then
    local anomalias = redis.call('HINCRBY', 'alarma:anomalias', vendor, 1)
    if anomalias % umbral == 0 then
        redis.call('HINCRBY', 'alarma:alertas', vendor, 1)
        return 1
    end
end
return 0
```

TODO 5 (`escrituras.py`)

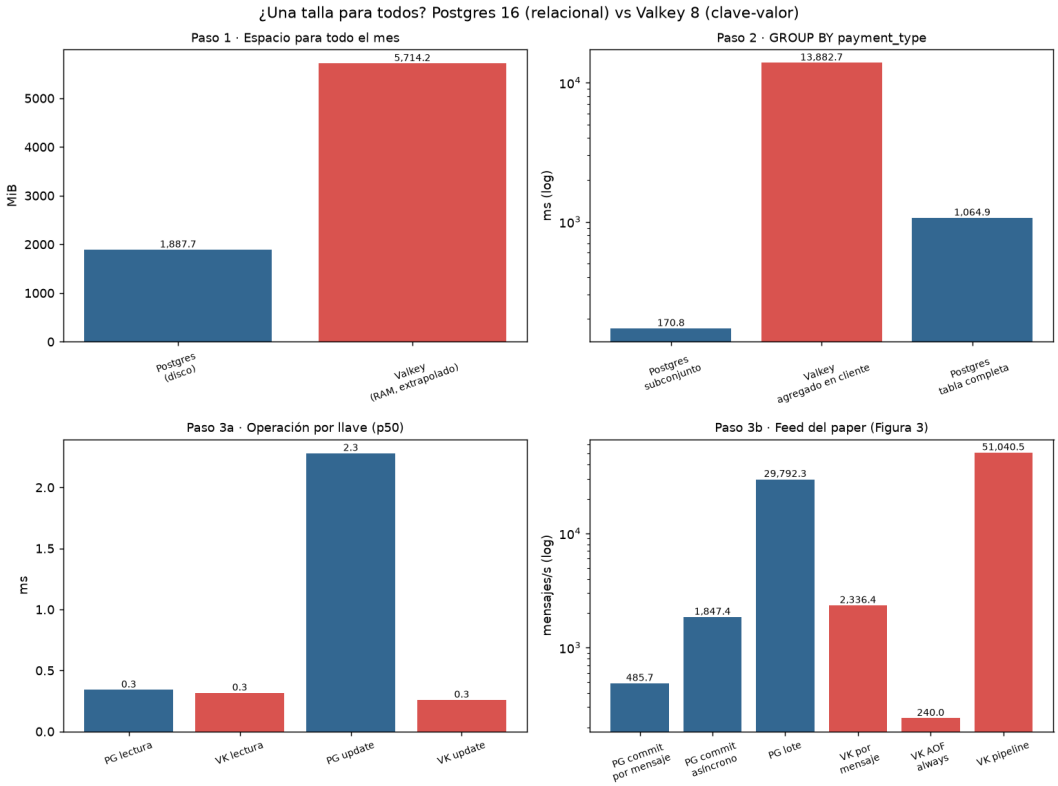
```
    for i in range(0, len(msgs), batch):
        with conn.transaction(), conn.cursor() as cur:
            cur.executemany(INSERT, [m[:4] for m in msgs[i:i + batch]])
```

Verificaciones (`evidence/salida_cargar.txt`, `salida_analitica.txt` y `salida_escrituras.txt`)

```
OK: TODO 1: total >= heap + índices, y ambos > 0
OK: TODO 2: Valkey tiene 1,000,000 llaves trip:*
OK: TODO 2: el último viaje tiene el campo fare_amount
OK: TODO 3: el GROUP BY a mano da los mismos conteos y promedios que Postgres
OK: postgres_commit_por_mensaje da la respuesta esperada
OK: postgres_commit_asincrono da la respuesta esperada
OK: postgres_lote da la respuesta esperada (TODO 5)
OK: valkey_por_mensaje da la respuesta esperada (TODO 4)
OK: valkey_aof_always da la respuesta esperada (TODO 4)
OK: valkey_pipeline da la respuesta esperada (TODO 4)
```

¿Por qué el pipeline y el lote cambian el tiempo? Los dos atacan un costo fijo que se paga por cada envío y no por cada fila. Sin pipeline, cada `HSET` espera su respuesta antes de mandar el siguiente: 1 000 000 de viajes de ida y vuelta por la red; con lotes de 10 000 son 100. En el feed, `valkey_pipeline` pasa de 2 336 a 51 040 msg/s (22×) con exactamente el mismo trabajo en el servidor. El lote de Postgres (TODO 5) junta 1 000 mensajes por transacción: el trigger corre igual por cada fila, pero el WAL se espera en disco 10 veces y no 10 000, y `executemany` manda las filas sin esperar la respuesta de cada `INSERT`. Pasa de 486 a 29 792 msg/s (61×). `COPY` sigue la misma idea en la carga: un solo flujo de bytes y un solo commit, sin un `INSERT` por fila.

3. Resultados



3.1. Paso 1: carga y espacio

	Postgres	Valkey
Filas cargadas	10 906 858	1 000 000
Tiempo	66,0 s `COPY` + 10,0 s índice PK	52,2 s
Filas por segundo	165 323	19 141
Espacio medido	1 653,6 MiB tabla + 233,7 MiB índice	523,9 MiB de RAM
Bytes por fila	181,5	549,4
Mes completo	1 887,7 MiB de disco	5 714,2 MiB de RAM (extrapolado)

¿Qué tienen en común los dos métodos de carga? Los dos evitan mandar un comando por fila y esperar su respuesta. `COPY` manda el CSV como un solo flujo de bytes en una transacción y el servidor lo parsea; el pipeline junta 10 000 `HSET` y los manda juntos. Aun así Valkey cargó 8,6 veces más lento: con `COPY` Python solo pasa líneas de texto, y con Valkey tiene que armar cada `HSET` con 19 pares campo-valor.

¿Qué pasa si el TODO 1 falla después del `COPY`? La conexión usa autocommit, así que el `COPY`, la llave primaria y el `VACUUM` ya están confirmados. La tabla queda cargada, pero el script se detiene: no se registran las medidas de disco ni se carga Valkey. Al volver a correr, `pg_trips.sql` hace `DROP TABLE` y la carga se repite completa (unos 76 s).

Bytes por fila y espacio total. Valkey usa 549,4 B por viaje y Postgres 181,5 B (159 de la tabla y 22 del índice), 3,0 veces menos. Postgres guarda cada valor con su tipo y los nombres de columna una sola vez, en el esquema; Valkey guarda todo como texto y repite los 19 nombres de campo en cada viaje. El mes completo serían 5 714 MiB de RAM contra 1 888 MiB de disco, y no cabe en los 4 GB del Valkey del curso.

¿Qué motor podría guardar un año entero en una laptop? Postgres. Un año son unos 131 M de viajes: unos 22 GiB de disco en Postgres contra unos 67 GiB de RAM en Valkey. Esta laptop tiene 16 GB de RAM.

3.2. Paso 2: consulta de almacén

	Postgres	Valkey
`GROUP BY` sobre el mismo millón de viajes (mediana)	170,8 ms	13 882,7 ms
`GROUP BY` sobre la tabla completa (mediana)	1 064,9 ms	—
Pregunta nueva: propina promedio por hora	1 329,4 ms	—

Sobre las mismas filas Postgres fue 81 veces más rápido. Valkey no tiene `GROUP BY`: el cliente recorrió las llaves con `SCAN`, trajo dos campos de cada viaje con `HMGET` y sumó en Python (el TODO 3 es el `HashAggregate` hecho a mano). La pregunta no anticipada se responde en Postgres con un `SELECT` nuevo sobre los mismos datos; en Valkey habría que escribir otro recorrido en el cliente.

3.3. Paso 3: escrituras y feed

3a, mediana de 2 000	Postgres	Valkey	Gana
Lectura por llave	0,341 ms	0,316 ms	Valkey, 1,08× (casi empate)
Update por llave	2,280 ms	0,258 ms	Valkey, 8,8×

La lectura es casi empate: en el servidor los dos hacen muy poco y lo que se mide es la red y el cliente. En el update, cada `UPDATE` de Postgres con autocommit espera a que el WAL llegue a disco con `fsync`; esos 1,9 ms de más son el precio de la D de ACID. Valkey cambia el valor en RAM y responde.

3b, 10 000 mensajes	msg/s	ms/msg	¿Espera al disco por mensaje?
`postgres_commit_por_mensaje`	486	2,059	Sí (WAL)
`postgres_commit_asincrono`	1 847	0,541	No
`postgres_lote`	29 792	0,034	Una vez por lote
`valkey_por_mensaje`	2 336	0,428	No
`valkey_aof_always`	240	4,167	Sí (AOF)
`valkey_pipeline`	51 040	0,020	No

Las seis variantes dan 7 alertas y los mismos conteos. Postgres procesa *outbound*: guarda el mensaje en `feed_trip` (versión de fila, heap y WAL) y después el trigger hace dos `INSERT ... ON CONFLICT`. Valkey procesa *inbound*: el mensaje no se guarda, pasa por `feed.lua` dentro del servidor y solo quedan cuatro contadores. Con el Count100 exacto del paper (50 000 mensajes, umbral 100) las conclusiones no cambian (`evidence/paso3.md`).

3.4. Tabla de medidas (evidence/tabla.md)

Paso	Motor	Métrica	Valor	Unidad
1-carga	postgres	bytes_por_fila	181.482	B/fila
1-carga	postgres	disco_indices	233.656	MiB
1-carga	postgres	disco_tabla	1,653.562	MiB
1-carga	postgres	disco_total	1,887.703	MiB
1-carga	postgres	filas	10,906,858	filas
1-carga	postgres	filas_por_s	165,323.443	filas/s
1-carga	postgres	tiempo_copy	65.973	s
1-carga	postgres	tiempo_indice_pk	9.982	s
1-carga	valkey	bytes_por_fila	549.356	B/fila
1-carga	valkey	filas	1,000,000	filas
1-carga	valkey	filas_por_s	19,140.676	filas/s
1-carga	valkey	ram_extrapolada_total	5,714.171	MiB
1-carga	valkey	ram_usada	523.906	MiB
1-carga	valkey	tiempo_carga	52.245	s
2-olap	postgres	explain_execution_time	1,445.133	ms
2-olap	postgres	mediana_completa	1,064.854	ms
2-olap	postgres	mediana_subconjunto	170.836	ms
2-olap	postgres	pregunta_nueva	1,329.447	ms
2-olap	valkey	agregado_en_cliente	13,882.670	ms
3a-puntual	postgres	lectura_por_llave_p50	0.341	ms
3a-puntual	postgres	update_por_llave_p50	2.280	ms
3a-puntual	valkey	lectura_por_llave_p50	0.316	ms
3a-puntual	valkey	update_por_llave_p50	0.258	ms
3b-feed	postgres_commit_asincrono	leer_respuesta	0.566	ms
3b-feed	postgres_commit_asincrono	mensajes_por_s	1,847.409	msg/s
3b-feed	postgres_commit_por_mensaje	leer_respuesta	0.921	ms
3b-feed	postgres_commit_por_mensaje	mensajes_por_s	485.707	msg/s
3b-feed	postgres_lote	leer_respuesta	1.166	ms
3b-feed	postgres_lote	mensajes_por_s	29,792.325	msg/s
3b-feed	valkey_aof_always	leer_respuesta	0.780	ms
3b-feed	valkey_aof_always	mensajes_por_s	239.968	msg/s
3b-feed	valkey_pipeline	leer_respuesta	0.895	ms
3b-feed	valkey_pipeline	mensajes_por_s	51,040.470	msg/s
3b-feed	valkey_por_mensaje	leer_respuesta	0.664	ms
3b-feed	valkey_por_mensaje	mensajes_por_s	2,336.417	msg/s

4. Plan de Postgres (evidence/plan_olap_postgres.txt)

```
Finalize GroupAggregate  (cost=292188.85..292189.91 rows=4 width=42) (actual time=1436.467..1443.935 rows=5 loops=1)
  Group Key: payment_type
  Buffers: shared hit=14 read=211656
  ->  Gather Merge  (cost=292188.85..292189.78 rows=8 width=42) (actual time=1436.392..1443.842 rows=13 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=14 read=211656
        ->  Sort  (cost=291188.82..291188.83 rows=4 width=42) (actual time=1420.062..1420.063 rows=4 loops=3)
              Sort Key: payment_type
              Sort Method: quicksort  Memory: 25kB
              Buffers: shared hit=14 read=211656
              Worker 0:  Sort Method: quicksort  Memory: 25kB
              Worker 1:  Sort Method: quicksort  Memory: 25kB
              ->  Partial HashAggregate  (cost=291188.73..291188.78 rows=4 width=42) (actual time=1420.039..1420.042 rows=4 loops=3)  <== nodo dominante
                    Group Key: payment_type
                    Batches: 1  Memory Usage: 24kB
                    Buffers: shared read=211656
                    Worker 0:  Batches: 1  Memory Usage: 24kB
                    Worker 1:  Batches: 1  Memory Usage: 24kB
                    ->  Parallel Seq Scan on trips  (cost=0.00..257103.27 rows=4544728 width=7) (actual time=0.046..459.170 rows=3635619 loops=3)
                          Buffers: shared read=211656  <== 211 656 x 8 KiB = 1 653,6 MiB = disco_tabla
Planning:
  Buffers: shared hit=39 read=7
Planning Time: 1.424 ms
JIT:
  Functions: 21
  Options: Inlining false, Optimization false, Expressions true, Deforming true
  Timing: Generation 2.210 ms, Inlining 0.000 ms, Optimization 1.800 ms, Emission 22.603 ms, Total 26.613 ms
Execution Time: 1445.133 ms
```

Leído de abajo hacia arriba:

Parallel Seq Scan. Dos workers más el proceso principal (`loops=3`), unas 3 635 619 filas cada uno, 10,9 M en total, en 459 ms. Leyó 211 656 páginas de 8 KiB: $211\,656 \times 8/1024 = 1\,653{,}6$ MiB, exactamente el `disco_tabla` del paso 1. Postgres leyó la tabla entera para responder.

Agregación en dos fases. Cada worker calcula su `Partial HashAggregate` por `payment_type`; `Gather Merge` junta los resultados parciales y `Finalize GroupAggregate` produce los 5 grupos finales.

Nodo dominante. El `Partial HashAggregate` con su escaneo toma 1 420 de los 1 445 ms (98 %): 459 ms de escaneo y el resto en agrupar y promediar 10,9 M filas. Todo el costo depende de cuántas filas y páginas se recorren, y aquí está el row-store: la consulta usa dos columnas (el escaneo entrega `width=7` bytes por fila), pero cada página trae las 20 columnas, unos 159 B por fila. Para usar dos columnas se leyeron 1 653,6 MiB. Eso es lo que un column-store evita.

5. Respuesta: ¿por qué «One Size Fits All» es una idea desfasada?

Stonebraker y Çetintemel parten de que los motores comerciales se diseñaron para el procesamiento de datos de negocio (OLTP) y que los fabricantes mantuvieron una sola línea de código para todo (sección 1). Su tesis es que esa estrategia ya falló en los almacenes de datos (sección 2) y que falla todavía más en el procesamiento de flujos (secciones 3 y 4), porque cada mercado necesita una arquitectura distinta (sección 7). El laboratorio repite la idea en pequeño: los mismos viajes de taxi en un motor relacional y en uno clave-valor, con tres cargas de trabajo distintas.

1. Ningún motor gana todas las cargas de trabajo. En el paso 1 gana Postgres: cargó el mes completo a 165 323 filas/s, 8,6 veces más rápido que Valkey (19 141 filas/s), y cada viaje ocupa 181 B en disco contra 549 B en RAM, 3,0 veces menos. En el paso 2 también gana Postgres, por 81× (punto 2). En el paso 3 gana Valkey: el update por llave le toma 0,26 ms contra 2,28 ms de Postgres (8,8×), y en el feed `valkey_pipeline` procesa 51 040 mensajes/s, 1,7 veces más que la mejor variante de Postgres (`postgres_lote`, 29 792) y 105 veces más que un commit por mensaje (486). La lectura por llave queda casi empatada (0,32 contra 0,34 ms).

Los números también muestran las limitantes de Valkey. Todo tiene que caber en RAM: el mes completo necesitaría 5 714 MiB, que no caben en los 4 GB que tiene asignados, y un año serían unos 67 GiB contra 22 GiB de disco en Postgres. No tiene tipos ni esquema, así que guarda todo como texto y repite los 19 nombres de campo en cada viaje. No tiene `GROUP BY`. Y su ventaja en el paso 3 depende de no escribir a disco: con `appendfsync always` baja a 240 mensajes/s. Cada motor gana donde su diseño calza con la carga.

2. Que Valkey pierda el `GROUP BY` no refuta el argumento del data warehouse. En el paso 2 Postgres respondió el `GROUP BY payment_type` sobre el mismo millón de viajes en 171 ms y Valkey en 13 883 ms. Podría parecer que esto contradice al paper, que dice que un row-store no es lo mejor para un almacén. No es así, por dos razones.

Primero, el experimento no enfrenta un row-store contra un column-store, sino un motor que tiene la primitiva de agregación contra uno que no la tiene. Valkey solo sabe responder «dame el valor de esta llave»: el cliente tuvo que recorrer las llaves con `SCAN`, traer dos campos de cada viaje con `HMGET` y sumar en Python. Postgres agregó dentro del motor, en paralelo y en dos fases (`Partial HashAggregate` y `Finalize GroupAggregate`). Es el argumento de la sección 4.2: sin la primitiva correcta en el motor se pierde, sin importar cómo se guarden los datos.

Segundo, el plan de Postgres muestra justo el costo que el paper le atribuye al row-store. El escaneo y la agregación parcial toman el 98 % del tiempo, y el escaneo leyó 211 656 páginas, 1 653,6 MiB: la tabla entera, con sus 20 columnas, para usar dos. Un column-store leería solo esas dos columnas, que es la ventaja que describe la sección 5.1 para consultas que agregan pocos atributos de muchas filas. Lo que sí confirma el paso 2 es la sección 2: Postgres respondió en 1 329 ms una pregunta que nadie previó (la propina promedio por hora), mientras que Valkey solo tiene lo que se guardó para responder por llave.

3. La brecha no es solo el overhead de `fsync()`. En el paper, StreamBase procesó 160 000 mensajes por segundo y el RDBMS 900 (sección 4). La explicación fácil es que el RDBMS escribe a disco y el motor de flujos no. Con los dos pares del paso 3b se puede separar qué parte es durabilidad y qué parte es modelo.

En el par con durabilidad (`valkey_aof_always` contra `postgres_commit_por_mensaje`) los dos esperan al disco en cada mensaje, y gana Postgres: 486 contra 240 msg/s (2,0×). El `fsync` se come el 74 % del tiempo de Postgres y el 90 % del de Valkey. Cuando los dos garantizan la D en cada mensaje, gana la bitácora mejor optimizada, no el modelo en memoria.

En el par sin durabilidad (`valkey_por_mensaje` contra `postgres_commit_asincrono`) ninguno espera al disco, y gana Valkey: 2 336 contra 1 847 msg/s (1,26×). Esa diferencia ya no es `fsync`, es el modelo. Postgres procesa outbound: guarda el mensaje en `feed_trip` y luego el trigger hace dos upserts. Valkey procesa inbound: el mensaje pasa por un script Lua dentro del servidor y solo quedan los contadores (secciones 4.1 y 4.3). Con la red repartida en lotes de 1 000, la ventaja del modelo sube a 1,7× (`valkey_pipeline` 51 040 contra `postgres_lote` 29 792).

Así, de la brecha de 105× entre `postgres_commit_por_mensaje` y `valkey_pipeline`, quitar el `fsync` explica solo 3,8×; agrupar los mensajes en lotes explica 16×, y el modelo deja 1,7× que no desaparece. El paper también explica por qué StreamBase no necesita ese `fsync`: logra alta disponibilidad con réplicas en vez de una bitácora en disco (sección 4.4) y no necesita transacciones ACID completas (sección 4.5).

Conclusión. Los tres pasos muestran lo mismo que el paper: el diseño que gana una carga pierde otra. El row-store con transacciones guarda, carga y agrega mejor; el motor en memoria que procesa el mensaje adentro gana las operaciones por llave y el feed, pero solo mientras no tenga que guardar nada. Para hacerlo todo bien, un solo producto tendría que llevar varias arquitecturas por dentro, que es justo lo que el paper llama el fin de «one size fits all» (sección 7).